

FLOWDESK

Project Report

NAME : TANMAY

PROJECT TYPE : Full-Stack Web Application (PERN Stack)

LIVE FRONTEND : <https://react-auth-frontend-liart.vercel.app>

LIVE BACKEND API : <https://trello-backend-1dsq.onrender.com>

FRONTEND GITHUB : github.com/JuniorTechie-coder/react-auth-frontend

BACKEND GITHUB : github.com/JuniorTechie-coder/trello-backend

INDEX

Sr.No	Title	PgNo
1	Introduction	3
2	Abstract	4
3	Technologies Used	5
4	Database Design	7
5	ER Diagram	9
6	Backend Architecture	10
7	Frontend Architecture	13
8	API Endpoints	15
9	Authentication System	17
10	Deployment	19
11	Current Status	21
12	Future Scope	22
13	Acknowledgement	23
14	Bibliography	24

INTRODUCTION

In today's fast-paced development environment, teams need a centralized, reliable, and intuitive platform to manage tasks, coordinate assignments, and track the overall progress of a project. Without such a system, teams often suffer from scattered task information, missed deadlines, unclear accountability, and poor visibility into the work pipeline.

FlowDesk is a full-stack, web-based project management application inspired by Trello. It allows individuals and teams to register, create workspaces, organize work into boards, break down work into lists and cards, and assign specific tasks to team members, all within a clean and interactive interface. The application bridges modern frontend and backend technologies to create a seamless, real-world project management experience.

The core objective of the project is to design and develop a production-ready, full-stack PERN (PostgreSQL, Express.js, React, Node.js) project management application with secure authentication, full CRUD capabilities, drag-and-drop task management, backlog tracking, and team assignment features.

Problem Statement

Development teams and students working on collaborative projects often lack a structured platform to plan and track work. Traditional tools like spreadsheets or simple to-do lists fail to represent task relationships, assignees, and workflow stages. FlowDesk solves this by introducing a hierarchical data model (User to Workspace to Board to List to Card) that maps directly to real project management workflows.

Key Application Goals

- **Secure Authentication** : JWT-based login and registration with bcrypt password hashing to protect user credentials.
- **Kanban-Style Task Management** : Visual boards with drag-and-drop cards across multiple workflow stages (lists).
- **Team Collaboration** : Assign cards to team members, filter by assignee, and provide role-based views.
- **Backlog Management** : A dedicated backlog panel for unplanned tasks, with the ability to promote them into active lists.
- **Customizable Themes** : Multiple board themes (Sunset, Magenta, Ocean, Midnight) stored in localStorage.
- **Full Deployment** : Frontend deployed on Vercel, backend API on Render, both live and production-ready.

ABSTRACT

FlowDesk is a web-based application developed to provide a centralized platform for creating, organizing, and tracking tasks within a collaborative team environment. In modern workflows, timely updates and clear task delegation are critical for improving coordination. The proposed system addresses these requirements by combining workspace management, interactive Kanban-style boards, task assignments, and role-based filtering within a single application.

The system allows users to register and log in securely using JWT (JSON Web Token) authentication with passwords encrypted using bcrypt. Once authenticated, users can create Workspaces, add Boards to them, and organize tasks into Lists and Cards. The application provides detailed card management, including drag-and-drop capability to move cards between lists or to and from a backlog. Team leads can utilize the backlog interface to plan tasks before moving them into active development cycles.

The project uses React 19 and Vite for the frontend, ensuring a fast, component-based, and responsive user experience with a custom navigation system implemented without third-party routing libraries. Node.js and Express.js are used for the backend to handle all RESTful API requests and business logic, while PostgreSQL is utilized for reliable relational database management, enforcing data integrity through foreign key relationships across all entities.

Authorization is enforced at every API endpoint. Controllers check whether the authenticated user owns the workspace, board, list, or card being accessed, preventing unauthorized data manipulation. The system is deployed and accessible publicly, with the frontend hosted on Vercel and the backend API on Render.

Architecture flow : React Frontend -> REST API (Express) -> JWT Middleware -> Controller -> PostgreSQL

TECHNOLOGIES USED

3.1 HTML5

HTML5 is the backbone of the frontend structure. It is used to define the layout and semantic elements of all components rendered by React, including forms, buttons, headings, containers, and lists.

3.2 CSS3

CSS3 is used to design the full visual interface, including board themes (Sunset, Magenta, Ocean, Midnight), card layouts, responsive containers, animations, and the global design system in index.css.

3.3 JavaScript (ES6+)

JavaScript is the primary programming language across both frontend and backend. It is used for component logic, API calls via `fetch()`, event handling, drag-and-drop, `localStorage`, and all server-side controllers.

3.4 React 19

React is a JavaScript library for building component-based user interfaces. FlowDesk uses React for all pages: Login, Register, Landing, Workspace, Board, and BoardDetail. State is managed with `useState` and `useEffect` hooks.

3.5 Vite 8

Vite is a next-generation frontend build tool used to scaffold and run the React application. It provides an extremely fast development server with Hot Module Replacement (HMR) and optimized production builds.

3.6 Node.js (v20)

Node.js is the JavaScript runtime that powers the backend server. It allows the use of JavaScript for server-side scripting, enabling a unified language across the entire stack. All API route handlers and controllers run in the Node.js runtime.

3.7 Express.js v5

Express.js is a minimal web framework for Node.js. It is used to define all REST API routes (`/api/auth`, `/api/workspaces`, `/api/boards`, `/api/lists`, `/api/cards`), apply middleware, and handle errors.

3.8 PostgreSQL

PostgreSQL is a powerful open-source relational database. It is used to persist all application data including users, workspaces, boards, lists, and cards. The `pg` library (`node-postgres`) with a connection pool is used to communicate with the database.

3.9 JSON Web Tokens (JWT)

JWT is used for stateless, token-based authentication. Upon successful login, a JWT signed with a secret key is returned to the client. It is sent in the `Authorization: Bearer <token>` header on all protected requests.

3.10 Bcrypt

Bcrypt is a password-hashing library used during user registration to hash passwords before storing them in the database. During login, `bcrypt.compare()` verifies the password against the stored hash securely.

3.11 Dotenv

Dotenv is used to manage environment variables (database connection string, JWT secret, port) through a `.env` file. It keeps sensitive configuration out of source code and enables easy configuration across environments.

3.12 Git and GitHub

Git and GitHub are used for version control throughout the project. Two separate repositories are maintained, one for the backend and one for the frontend, enabling independent deployment and development workflows.

DATABASE DESIGN

FlowDesk uses PostgreSQL as its relational database. The schema is designed around a strict hierarchical ownership model that enforces data integrity at every level using foreign key constraints. Every query in the controllers validates ownership by joining through the full chain of relationships back to the workspaces table and checking owner_id.

Core Tables

Table	Key Columns	Description
users	user_id, name, email, password	Stores registered users. Passwords are bcrypt-hashed before storage.
workspaces	id, name, description, owner_id	Groups of boards owned by a user. owner_id references users.user_id.
boards	id, name, workspace_id	Kanban boards inside a workspace. workspace_id references workspaces.id.
lists	id, name, position, board_id	Columns within a board representing workflow stages (e.g. To Do, In Progress, Done).
cards	id, title, description, position, list_id, board_id, is_backlog, assigned_to, created_by, updated_at	The primary task unit. Cards can belong to a list or be in the backlog. They can be assigned to a user.

Ownership Chain Enforcement

A key security design in FlowDesk is that every sensitive query joins back to verify the authenticated user owns the resource. Below is an example from cardController.js, showing the ownership verification query used before updating a card.

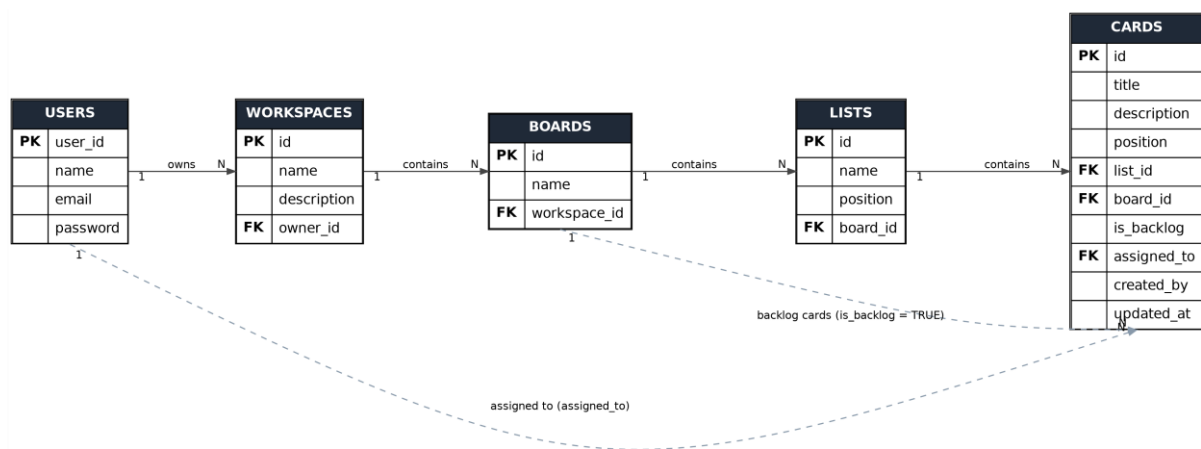
```
// Before updating a card, verify full ownership chain
const cardResult = await pool.query(
  `SELECT cards.*, COALESCE(cards.board_id, lists.board_id) AS current_board_id
  FROM cards
  LEFT JOIN lists ON cards.list_id = lists.id
  LEFT JOIN boards ON (cards.board_id = boards.id OR lists.board_id = boards.id)
  JOIN workspaces ON boards.workspace_id = workspaces.id
  WHERE cards.id = $1
  AND workspaces.owner_id = $2`,
  [id, req.user.user_id] // req.user comes from JWT middleware
);
```

Card Backlog Field

The cards table includes an is_backlog boolean column. When is_backlog is TRUE, the card belongs to the board-level backlog and its list_id is set to NULL. This allows seamless movement of cards between the backlog and any list without creating additional tables.

ER DIAGRAM

The following diagram illustrates the entity relationships in the FlowDesk database. The model enforces a strict parent-child ownership hierarchy from User down to Card.



Cardinality Summary

Relationship	Type	Foreign Key
User -> Workspace	1 : N	workspaces.owner_id -> users.user_id
Workspace -> Board	1 : N	boards.workspace_id -> workspaces.id
Board -> List	1 : N	lists.board_id -> boards.id
List -> Card	1 : N	cards.list_id -> lists.id
User -> Card (assignment)	1 : N	cards.assigned_to -> users.user_id
Board -> Card (backlog)	1 : N	cards.board_id -> boards.id (when is_backlog=TRUE)

BACKEND ARCHITECTURE

Project Structure

```
flowdesk-backend/
|
|-- controllers/
|   |-- authController.js      # Register & Login logic
|   |-- workspaceController.js # CRUD for workspaces
|   |-- boardController.js     # CRUD for boards
|   |-- listController.js      # CRUD for lists
|   |-- cardController.js      # CRUD + assign + backlog for cards
|   |-- userController.js      # CRUD for users
|
|-- middleware/
|   |-- authMiddleware.js      # JWT verification, protects all routes
|
|-- routes/
|   |-- auth.js                # /api/auth
|   |-- workspaces.js          # /api/workspaces
|   |-- boards.js              # /api/boards
|   |-- lists.js               # /api/lists
|   |-- cards.js               # /api/cards
|   |-- users.js               # /api/users
|
|-- .env                       # PORT, DATABASE_URL, JWT_SECRET
|-- app.js                     # Express server entry point
|-- db.js                      # PostgreSQL Pool connection
|-- package.json
```

Main Server Entry Point (app.js)

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const pool = require('./db');

// Route imports
const userRoutes = require('./routes/users');
const workspacesRoutes = require('./routes/workspaces');
const boardsRoute = require('./routes/boards');
const listsRoute = require('./routes/lists');
const cardsRoute = require('./routes/cards');
const authRoute = require('./routes/auth');

const app = express();
app.use(express.json());
app.use(cors());

// Request logger middleware
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
});

// Route mounting
app.use('/api/users', userRoutes);
app.use('/api/workspaces', workspacesRoutes);
app.use('/api/boards', boardsRoute);
app.use('/api/lists', listsRoute);
app.use('/api/cards', cardsRoute);
app.use('/api/auth', authRoute);

// 404 fallback
app.use((req, res) => res.status(404).json('error: Route not found'));
```

```
const PORT = process.env.PORT || 3000;  
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Database Connection (db.js)

```
require('dotenv').config();  
const { Pool } = require('pg');  
  
const pool = new Pool({  
  connectionString: process.env.DATABASE_URL,  
  ssl: { rejectUnauthorized: false } // Required for Render/Supabase hosted DBs  
});  
  
module.exports = pool;
```

FRONTEND ARCHITECTURE

Project Structure

```
flowdesk-frontend/src/
|
|-- pages/
|   |-- Landing.jsx      # Marketing / home page
|   |-- Login.jsx        # Login form with JWT storage
|   |-- Register.jsx     # Registration form
|   |-- Workspace.jsx    # Workspace listing and management
|   |-- Board.jsx        # Board listing within a workspace
|   |-- BoardDetail.jsx  # Full Kanban board (lists + cards + backlog)
|
|-- App.jsx              # Root component, routing logic
|-- navigation-context.jsx # Custom SPA navigation (no React Router)
|-- toast-context.js     # Global toast notification context
|-- toast.jsx            # Toast notification component
|-- ErrorBoundary.jsx    # React Error Boundary wrapper
|-- index.css            # Global design system styles
|-- main.jsx             # ReactDOM render root
```

Custom Navigation System

FlowDesk implements its own lightweight SPA navigation using React Context (navigation-context.jsx) instead of React Router. A `currentPath` state variable and a `navigateTo()` function update the browser URL using the History API and re-render the correct page component from `App.jsx`.

Application Routing Logic (App.jsx)

```
function App() {
  const { currentPath, navigateTo } = useNavigation();
  const [isLoggedIn, setIsLoggedIn] = useState(() =>
    !!localStorage.getItem("token"));

  // Authenticated routing
  if (isLoggedIn) {
    if (currentPath.startsWith("/board/")) return <BoardDetail
handleLogout={handleLogout} />;
    if (currentPath.startsWith("/boards/")) return <Board
handleLogout={handleLogout} />;
    if (currentPath === "/workspace") return <Workspace handleLogout={handleLogout}
/>;
  }

  // Unauthenticated routing
  if (isLoginView) return <Login setIsLoggedIn={...} />;
  if (isRegisterView) return <Register />;
  return <Landing />;
}
```

Key Frontend Features

- **Drag and Drop (Native HTML5)** : Cards can be dragged between lists and to/from the backlog using native HTML5 drag events (`onDragStart`, `onDrop`, `onDragOver`). The backend is updated via `PUT /api/cards/:id` with the new `list_id`, `board_id`, and `is_backlog` values.

- Backlog Panel : A dedicated side panel in BoardDetail.jsx shows backlog cards. Team leads can create backlog cards with `is_backlog: true`, and then promote them to any list by updating the card.
- Assignee Filtering : Users can filter cards on a board by All Tasks, My Tasks, Unassigned, or a specific team member, enabling role-based views without server-side filtering.
- Board Themes : Four visual themes are available: Sunset, Magenta, Ocean, Midnight. The selected theme is applied as a CSS class on the board container and persisted in `localStorage`.

API ENDPOINTS

All endpoints except `/api/auth/*` require a valid Authorization: Bearer <token> header. The protect middleware validates the JWT on every protected route.

Authentication

Method	Endpoint	Description	Auth?
POST	<code>/api/auth/register</code>	Register a new user, hash password, return user info	No
POST	<code>/api/auth/login</code>	Authenticate user, verify bcrypt, return JWT + user info	No

Users

Method	Endpoint	Description
GET	<code>/api/users</code>	Get all users
GET	<code>/api/users/:id</code>	Get a specific user by ID
POST	<code>/api/users</code>	Create a new user
PUT	<code>/api/users/:id</code>	Update a user's information
DELETE	<code>/api/users/:id</code>	Delete a user

Workspaces

Method	Endpoint	Description
GET	<code>/api/workspaces</code>	Get all workspaces owned by the logged-in user
GET	<code>/api/workspaces/:id</code>	Get a specific workspace by ID
POST	<code>/api/workspaces</code>	Create a new workspace
PUT	<code>/api/workspaces/:id</code>	Update workspace name or description
DELETE	<code>/api/workspaces/:id</code>	Delete a workspace

Boards

Method	Endpoint	Description
GET	<code>/api/boards/workspace/:id</code>	Get all boards for a specific workspace
GET	<code>/api/boards/:id</code>	Get a single board by ID
POST	<code>/api/boards</code>	Create a new board inside a workspace
PUT	<code>/api/boards/:id</code>	Update a board
DELETE	<code>/api/boards/:id</code>	Delete a board

Lists

Method	Endpoint	Description
GET	<code>/api/lists/board/:id</code>	Get all lists for a specific board

Method	Endpoint	Description
POST	/api/lists	Create a new list inside a board
PUT	/api/lists/:id	Update a list's name or position
DELETE	/api/lists/:id	Delete a list (cascades to cards)

Cards

Method	Endpoint	Description
GET	/api/cards/lists/:id	Get all non-backlog cards for a specific list
GET	/api/cards/:id	Get a single card by ID (includes assignee info)
GET	/api/cards/board/:boardId/backlog	Get all backlog cards for a board
POST	/api/cards	Create a card (list card or backlog card)
PUT	/api/cards/:id	Update card (title, desc, position, move to list/backlog)
PUT	/api/cards/:id/assign	Assign or unassign a card to a team member
DELETE	/api/cards/:id	Delete a card permanently

AUTHENTICATION SYSTEM

FlowDesk uses a stateless JWT-based authentication system. No sessions are stored on the server. The client stores the token in localStorage and sends it on every protected request.

Registration Flow

Client sends name, email, password -> Validation check -> bcrypt.hash(password, 10) -> INSERT INTO users -> Return user_id, name, email.

```
const authRegistration = async (req, res) => {
  const { name, email, password } = req.body;
  if (!name || !email || !password)
    return res.status(400).json({ error: 'All fields are required!' });

  const hashedPassword = await bcrypt.hash(password, 10);
  const result = await pool.query(
    'INSERT INTO users(name, email, password) VALUES($1, $2, $3) RETURNING name, email, user_id',
    [name, email, hashedPassword]
  );

  res.status(201).json({ message: 'User registered successfully!',
    ...result.rows[0] });
  // Error code 23505 = unique email violation -> 409 Conflict
};
```

Login Flow

Client sends email, password -> Find user by email -> bcrypt.compare() -> jwt.sign() with 7 day expiry -> Return token + user info.

```
const isMatch = await bcrypt.compare(password, findUser.rows[0].password);
if (!isMatch) return res.status(401).json({ error: 'Invalid email or password!' });

const token = jwt.sign(
  { user_id: user.user_id, email: user.email }, // JWT payload
  process.env.JWT_SECRET, // Signing secret from .env
  { expiresIn: '7d' } // Token valid for 7 days
);

res.status(200).json({ message: 'Login Successful!', token, user_id, name, email
});
```

JWT Middleware (Route Protection)

```
const protect = async (req, res, next) => {
  const authHeader = req.headers.authorization;
  const token = authHeader?.split(' ')[1]; // "Bearer eyJ..." -> "eyJ..."
  if (!token) return res.status(401).json({ error: 'No token, access denied!' });

  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.user = decoded; // Attaches { user_id, email } to the request object
  next(); // Passes to the controller
};
```

DEPLOYMENT

FlowDesk is deployed as two separate services, the frontend on Vercel and the backend API on Render. Both are publicly accessible and connected to each other.

Item	Value
Frontend Platform	Vercel (free tier)
Backend Platform	Render (free tier)
Live Frontend URL	https://react-auth-frontend-liart.vercel.app
Live Backend API	https://trello-backend-1dsq.onrender.com
Database	PostgreSQL (hosted via Render or Supabase)

Frontend Deployment (Vercel)

The React and Vite application is built with `npm run build`, producing a `dist/` folder. Vercel auto-deploys on every push to the connected GitHub repository. A `vercel.json` file configures SPA rewrite rules so that direct URL access works correctly.

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/" }]
}
```

Backend Deployment (Render)

The Express.js server is deployed on Render as a Web Service. Environment variables (`DATABASE_URL`, `JWT_SECRET`, `PORT`) are configured through Render's dashboard. The server listens on the `PORT` provided by Render at runtime.

```
PORT=3000
DATABASE_URL=postgresql://user:password@host/dbname
JWT_SECRET=your_super_secret_key
```

Local Development Setup

```
# Backend
git clone https://github.com/JuniorTechie-coder/trello-backend.git
cd trello-backend
npm install
# Create .env with PORT, DATABASE_URL, JWT_SECRET
node app.js

# Frontend
git clone https://github.com/JuniorTechie-coder/react-auth-frontend.git
cd react-auth-frontend
npm install
npm run dev
```

CURRENT STATUS

Completed Features

- PostgreSQL database setup with full relational schema
- User registration with bcrypt password hashing
- User login with JWT token generation (7-day expiry)
- JWT middleware protecting all private API routes
- CRUD operations for Workspaces
- CRUD operations for Boards
- CRUD operations for Lists
- CRUD operations for Cards
- Card assignment to team members (PUT /api/cards/:id/assign)
- Backlog management (create, view, promote backlog cards)
- Drag-and-drop card movement between lists
- Move cards between list and backlog
- Assignee filtering (All / My Tasks / Unassigned / Specific member)
- Multiple board visual themes (Sunset, Magenta, Ocean, Midnight)
- CORS configuration for cross-origin API requests
- Postman API testing
- React frontend with full backend integration
- Frontend deployed on Vercel
- Backend API deployed on Render
- Custom SPA navigation (no React Router dependency)
- Global toast notification system
- Error boundary implementation

In Progress

- Improving form validation and error messages
- Improving responsive design for mobile screens
- Further UI/UX polish

FUTURE SCOPE

FlowDesk can be further enhanced by implementing resource-level authorization, so that team members can be invited to specific workspaces and boards with defined roles such as viewer, editor, and admin, rather than the current single-owner model.

Real-time collaboration can be added by integrating WebSockets (Socket.io), so that when one user moves a card, creates a list, or assigns a task, all other users on the same board see the update instantly without refreshing the page.

Automated testing can be introduced using Jest and Supertest for the backend API, and React Testing Library for the frontend components, to ensure long-term reliability.

The platform can be extended to a native mobile application using React Native, allowing users to manage tasks, view boards, and receive push notifications from iOS and Android devices.

An analytics dashboard can be built to show charts for task completion rates, workload per team member, overdue cards, and sprint velocity, giving project managers better visibility into team performance.

A due date field can be added to cards along with automated email or in-app reminders when deadlines are approaching, helping teams avoid missed deliverables.

Card comments can be introduced to allow team members to leave threaded comments on cards for discussions, clarification, and progress updates, reducing the need for external communication tools.

Third-party integrations with tools like GitHub (linking PRs to cards), Slack (notifications), and Google Calendar (syncing due dates) can help embed FlowDesk into existing developer workflows.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported and guided us throughout the development of the FlowDesk project.

We are thankful to our project guide and faculty members for providing us with the necessary knowledge, direction, and support during the different stages of the project. Their constructive feedback and reviews significantly helped us improve our understanding of full-stack web development, database management, REST API design, and authentication systems.

We are grateful to the creators and contributors of the open-source tools and frameworks that made this project possible, including the teams behind React, Node.js, Express.js, PostgreSQL, Vite, and the many npm packages used throughout the development process. Their freely available documentation, tutorials, and community support were invaluable learning resources.

We appreciate the platforms Vercel and Render for providing free-tier deployment services that allowed the project to be hosted, tested, and shared in a real production environment.

We are grateful to our friends and classmates for their valuable suggestions, feedback during testing, and encouragement throughout the development process.

Finally, we would like to thank our family members for their constant support, patience, and motivation which kept us going throughout the development of this project.

BIBLIOGRAPHY

- MDN Web Docs — HTML, CSS, JavaScript, Fetch API, Drag & Drop API reference (developer.mozilla.org).
- React Documentation — Hooks (useState, useEffect, useContext), Components, JSX (react.dev).
- Node.js Documentation — Runtime APIs, modules, HTTP (nodejs.org/docs).
- Express.js Documentation — Routing, middleware, error handling (expressjs.com).
- PostgreSQL Documentation — SQL queries, JOINS, foreign keys, constraints (postgresql.org/docs).
- node-postgres (pg) Docs — Pool configuration, parameterized queries (node-postgres.com).
- JWT.io — JSON Web Token standard, payload structure, verification (jwt.io).
- bcrypt npm package — Password hashing and comparison (npmjs.com/package/bcrypt).
- Vite Documentation — Dev server, build configuration, HMR (vitejs.dev).
- Vercel Documentation — Frontend deployment, vercel.json rewrites (vercel.com/docs).
- Render Documentation — Web service deployment, environment variables (render.com/docs).
- GitHub Docs — Repository management, version control, collaboration (docs.github.com).
- Postman — API testing and documentation tool (postman.com).
- OpenAI ChatGPT — Used as an AI-assisted learning and development resource for understanding concepts, troubleshooting code, debugging errors, and improving the implementation of the project.